


```

If (a[0] == b[0]) c[0] = 1; else c[0] = 0;
If (a[1] == b[1]) c[1] = 1; else c[1] = 0;
...
If (a[15] == b[15]) c[15] = 1; else c[15] = 0;

If (c[0:15] == 1) y[0:15] = a[0:15]; else y[0:15] = 0;

```

즉, source는 a와 b의 모든 비트가 같을 때(=c가 all-1일 때) a를 y로 그대로 내보내고, 아니면 0을 내보내는 회로이다. Destination은 select signal s[0:3]을 받아 y[0:15] 중 하나의 bit를 최종 data D로 출력하는 **16:1 MUX**이며, 출력단에는 100 fF의 load cap이 달린다. 설계한 회로는 logical effort로 예측한 delay와 simulation을 통해 추출한 delay를 비교 검증하였다.

설계 조건

- 조건 1) 위의 두 가지 조건문을 만족하는 회로를 설계한다.
- 조건 2) 설계하는 source는 c[0:15]를 첫 stage의 input으로 한다.
- 조건 3) Path delay와 area를 optimize 하도록 설계한다. (stage 수 조절 가능)
- 조건 4) 1단계에서는 bus metal line의 RC를 무시하고 destination 단의 gate capacitance만을 고려하여 설계한다.
- 조건 5) Source의 첫 번째 gate는 unit-sized inverter (NMOS: 1.1 μm , PMOS: 2.2 μm)와 동일한 driving current를 갖도록 sizing 한다.
- 조건 6) Logical effort 계산 시, gate 및 drain cap으로는 직접 측정한 값을 사용한다.
- 조건 7) Select signal은 s[0:3]을 사용하고 조합에 따라 D는 진리표와 같은 결과를 출력한다.
- 조건 8) 16:1 mux는 다양한 stage 수와 gate 종류로 구성할 수 있고 path delay와 area를 optimize 하도록 설계한다. (logic gate, 2:1 mux, 4:1 mux, 8:1 mux 조합 가능)
- 조건 9) 주어진 load cap 값 (100 fF)을 이용해 설계한다.

진행 순서: ① mux 설계는 metal line RC를 고려하지 않고 먼저 진행 → ② metal line 설계 (최적 위치 repeater 삽입 + capacitive crosstalk 분석) → ③ metal line RC를 반영한 16:1 mux 재설계 및 estimated/simulated delay 비교.

1. Design Basic Data Estimation

회로 설계에 앞서, logical effort 기반 delay 예측에 필요한 기초 데이터(단위 소자의 gate/drain capacitance, mobility 비율, FO1/FO4 delay)를 시뮬레이션으로 직접 측정하였다. 수업에서는 gate capacitance로 normalize 된 unit-less 값을 사용했지만, 실제 metal cap을

절대량으로 고려하기 위해서는 unit inverter의 gate cap을 fF 단위로 측정해 둘 필요가 있다.

1.1 Gate Capacitance 측정

측정 원리 — FO1 input shaping을 위해 unit inverter(PMOS 2.2 μm / NMOS 1.1 μm) 3개를 직렬로 연결한 chain을 두 벌 만든다. 한 벌의 출력에는 측정 대상 MOSFET(NMOS는 Drain(Source)·Bulk을 VSS로, PMOS는 Drain(Source)·Bulk을 VDD로 연결하고 gate를 출력 노드에 연결)을 달고, 다른 벌의 출력에는 이상적인 capacitor를 단다. Capacitor 값을 sweep 하면서 **두 출력 노드의 파형 slope(rising, falling)이 같아지는 cap 값을** 찾으면, 그 값이 해당 gate 폭($3.3 \mu\text{m} = 1.1 + 2.2$)에 해당하는 gate capacitance이다.

측정 결과 — capacitor 1 fF~5 fF 5종에 대해 rising/falling delay를 각 5회씩 측정하였다. t_{pd} 는 $V_{DD}/2$ 기준이기 때문에 **550 mV**를 기준으로 설정하였다.

표 1.2 t_{pdr} , t_{pdf} 측정 결과 (capacitor) [단위: ps]

Delay	1	2	3	4	5	average
1fF rising	10.9	10.9	11.1	10.9	11.1	10.98
1fF falling	11.8	12.0	12.0	12.0	12.0	11.96
2fF rising	12.1	12.2	12.3	12.1	12.2	12.18
2fF falling	13.2	13.2	13.2	13.3	13.3	13.24
3fF rising	13.2	13.2	13.3	13.2	13.3	13.24
3fF falling	14.4	14.4	14.5	14.5	14.6	14.48
4fF rising	14.2	14.4	14.3	14.2	14.3	14.28
4fF falling	15.5	15.6	15.7	15.8	15.8	15.68
5fF rising	15.2	15.4	15.4	15.2	15.3	15.30
5fF falling	16.7	16.8	16.8	16.9	16.9	16.82

nMOS와 pMOS gate를 부하로 단 경우의 평균 delay는 표 1.1과 같았다.

표 1.1 MOS gate 부하 시 평균 propagation delay

부하	t _{pd} (ps)
nMOS gate (W = 1.1 μm)	11.967
pMOS gate (W = 2.2 μm)	13.570

표 1.2의 결과로 average propagation delay와 capacitance의 관계를 추세선으로 표시하고 (그림 1.6, 기울기 약 1.1 ps/fF, 절편 약 10 ps), 이 추세선에 gate 부하일 때의 delay를 대입하여 측정한 gate delay에 해당하는 capacitance 값을 구했다.

C_{inv(unit)} = 4.1216 fF (unit inverter, 총 gate width 3.3 μm 기준) → 폭으로 정규화하면 **1.249 fF/μm**

Gate cap은 gate width에 비례하므로, 이후 설계에서 특정 gate의 절대 cap이 필요할 때는 해당 width를 대입해 환산하였다.

1.2 Drain Capacitance 측정

앞서 측정한 gate capacitance와 비슷하게 이번엔 drain capacitance를 측정하였다. 해당 schematic은 각각 inverter 3개와 pMOS 1개로 이루어진 윗줄과, inverter 3개와 nMOS 1개로 이루어진 아랫줄로 구성되어 있다 (그림 1.7). 단, 이번에는 **gate에 신호를**

전달하지 않고 drain에 신호를 전달하여 delay를 측정하였다 (NMOS → VSS: Gate, Drain(Source), Bulk / PMOS → VDD: Gate, Drain(Source), Bulk).

그림 1.8과 같이 nMOS, pMOS의 rising, falling delay를 각각 5번씩 측정하여 평균낸 결과를 표 1.3에 정리하였고, 이를 통해 nMOS와 pMOS의 drain capacitance를 측정하였다.

표 1.3 t_r, t_f 측정 결과 (pmos, nmos) [단위: ps]

Delay	1	2	3	4	5	average
t _{pdr,n}	11.6	11.5	11.4	11.0	11.6	11.42
t _{pdf,n}	10.6	10.7	10.8	10.9	10.5	10.70
t _{pdr,p}	11.3	12.3	12.2	11.8	12.3	11.98
t _{pdf,p}	12.0	12.0	12.0	12.1	11.9	12.00

- t_{avg,n} = 11.060 ps
- t_{avg,p} = 11.990 ps

측정 결과:

- nMOS drain capacitance: $C_{d,n} = 0.5930 \text{ fF} \rightarrow \text{per width } 0.5391 \text{ fF}/\mu\text{m}$ ($W = 1.1 \mu\text{m}$)
- pMOS drain capacitance: $C_{d,p} = 1.405 \text{ fF} \rightarrow \text{per width } 0.6387 \text{ fF}/\mu\text{m}$ ($W = 2.2 \mu\text{m}$)

1.3 Mobility Comparison

이번에는 nMOS와 pMOS의 mobility를 측정하였다. 그림 1.9와 같이 inverter를 여러 개 직렬로 연결한 다음 마지막에 capacitor를 달아주었다. nMOS와 pMOS의 mobility 비율은 **PN ratio를 설정할 때 중요하게 이용된다.**

현재 사용하는 inverter는 nMOS와 pMOS의 length는 45 nm로 동일하고, width는 각각 1.1 μm , 2.2 μm 인 MOSFET으로 구성되어 있다. MOSFET에서 흐르는 전류는 다음의 식을 따른다.

$$I_d = \frac{1}{2} \frac{W}{L} \mu C_{ox} (V_{GS} - V_{th})^2$$

nMOS와 pMOS의 전류의 비를 **beta ratio**라고 하며, 이는 mobility, width 두 가지 요소에 의해서만 결정된다. 만약 nMOS와 pMOS가 동일한 strength를 가진다면 동일한 전압과 capacitor에서 동일한 delay를 가질 것이다. 즉, **inverter에서 rising time과 falling time이 동일해지는 것을** 찾으려 한다.

해당 논리를 바탕으로 먼저 현재 inverter의 rising, falling delay 차이를 확인하였다. rising delay와 falling delay는 V_{DD} 의 0.2배, 0.8배가 되는 때를 기준으로 하므로, **220 mV와 880 mV**를 기준으로 설정하여 delay를 측정하였다 (그림 1.10~1.13).

결과 — $W_p : W_n = 2 : 1$ (2.2 $\mu\text{m} : 1.1 \mu\text{m}$)에서 rising delay와 falling delay가 근사하게 일치함을 확인하였다. 이는 nMOS mobility가 pMOS 대비 약 2배임을 의미하며, 이 비율($\mu = 2$)을 이후 모든 gate의 PN sizing에 사용하였다.

1.4 FO1 & FO4 Delay

측정 결과 다음과 같이 FO1 delay와 FO4 delay를 구할 수 있었다.

- $t_{pd,FO1} = 15.04 \text{ ps}$
- $t_{pd,FO4} = 27.40 \text{ ps}$

FO delay 식 $t_{pd,FOh} = (h + p_{inv})\tau$ 에 $h = 1, 4$ 를 각각 대입하여 연립하면:

- $\tau = 4.121 \text{ ps}$
- $p_{inv} = 2.650$

FO4 delay를 측정하기 위해 동일하게 세 개의 inverter와 capacitor가 달린 회로를 3개 추가하였고, 네 번째 inverter의 뒷 단에서 동일하게 delay를 측정하였다 (그림 1.15~1.17). 이 τ 와 p_{inv} 값은 이후 logical effort로 계산한 정규화 delay D를 실제 ps 단위로 환산하는데 사용된다.

2. Source, Destination 설계

2.1 Outline

Source는 a, b, c를 input으로 받아 a[i]와 b[i]가 같으면 c[i]가 1이 되고, c가 모두 1일 때 a를 output으로 내보낸다. 그 외에는 output이 0이 되도록 하는 logic을 가진다. Bus는 그대로 source와 destination을 연결하여 output을 전달하는 역할을 하고, destination에서는 source로부터 전달 받은 16-bit의 값 중 하나를 선택하여 내보내는 역할을 한다. 즉, destination에서는 16-to-1 MUX가 필요하다.

그러므로, c가 모두 1의 bit를 가질 때 output을 y로 가지는 logic을 만들기 위해서 source 단에는 **1's detector**가 필요하고, 이 1's detector의 값이 1일 때 a[i]의 값을 선택하는 logic을 구성해야 한다. 더불어 destination에서는 16-to-1 MUX를 구성해야 한다.

2.2 MUX 구조 분석

MUX는 그림 2.2와 그림 2.3에서 볼 수 있듯 NAND gate들을 이용하여 설계하였다. MUX는 여러 가지 방법으로 설계할 수 있는데, 그 중 사용할 수 없는 transmission gate 등을 제외하였다. **Compound gate MUX를 사용하지 않은 이유는 2-to-1 MUX를 기준으로 생각했을 때 NAND2 gate를 사용한 MUX가 가장 delay가 작았기 때문이다.** 각 구조의 logical effort는 다음과 같이 계산하였다 ($\mu = 2$).

$$g_{com} = \frac{2\mu + 2}{\mu + 1} \times 1 = 2, p_{com} = \frac{4\mu + 4}{\mu + 1} \times 1 \times P_{inv} = 4P_{inv}$$

$$g_{tri} = \frac{2\mu + 2}{\mu + 1} \times 1 = 2, p_{tri} = \frac{4\mu + 4}{\mu + 1} \times 1 \times P_{inv} = 4P_{inv}$$

$$g_{nand2} = \left(\frac{\mu+2}{\mu+1} \right)^2 = 1.78, p_{nand2mux} = \left(\frac{2\mu+2}{\mu+1} \right)^2 \times P_{inv} = 4 P_{inv}$$

- 그림 2.2: **4-to-1 MUX with NAND3+NAND4** — (D0,S0',S1'), (D1,S0',S1), (D2,S0,S1'), (D3,S0,S1)을 각각 NAND3로 받고 그 출력 4개를 NAND4로 합쳐 Y 출력
- 그림 2.3: **2-to-1 MUX with NAND2+NAND2** — (D0,S0'), (D1,S0)을 NAND2로 받고 NAND2로 합쳐 Y 출력

이를 통해 NAND를 이용하여 만든 MUX가 가장 delay가 작은 것을 알 수 있었다. 이는 compound gate와 tristate buffer 모두 nmos, pmos transistor를 직렬로 배열해야 하기 때문에 transistor의 width가 증가하기 때문이다. 그러므로, input 개수가 늘어나도 여전히 NAND를 이용한 MUX가 더 빠를 것임을 예상할 수 있다.

2.3 Logic Gate별 Logical Effort, Parasitic Delay

Inverter를 포함하여 NAND2, NAND3, NAND4, NAND5, NOR2, NOR4, NAND16, MUX16, MUX4, MUX2에 대한 g와 p를 구하여 표 2.1에 나타내었다. MUX의 g·p는 구성 gate의 곱/합으로 계산하였다 (MUX2 = NAND2+NAND2, MUX4 = NAND3+NAND4, MUX16 = NAND5+NAND16).

표 2.1 Logic gate들의 logical effort, parasitic delay (MUX는 NAND3-NAND4 구조)

Delay	g	p
INV	1.000	2.65
NAND2	1.336	5.30
NAND3	1.672	7.95
NAND4	2.008	10.60
NAND5	2.344	13.25
NOR2	1.664	5.30
NOR4	2.992	10.60
NAND16	10.959	42.40
MUX16	25.691	55.65
MUX4	3.358	18.55
MUX2	1.785	10.60

2.4 Logic 후보 비교 및 Target Design 선정

이제 NAND16+NOR2+MUX16과 동일한 logic을 수행하는, 다양한 delay를 가지는 logic들을 비교한다. 총 15개의 후보 구조에 대해 B(branch), G(총 logical effort), H, F(=GBH), P(총 parasitic), N(stage 수), D(정규화 delay)를 계산하였다.

표 2.2 Logic 조합별 delay 계산 결과 ($D = N \cdot F^{1/N} + P$)

#	Logic	B	G	H	F	P	N	D
1	NAND16-NOR2-MUX16	16	468.47	2.21	16595	103.35	4	148.75
2	NAND4-NOR4-NAND2-INV-MUX16	16	206.23	12.08	39865	84.80	6	119.87
3	NAND4-NOR4-INV-NOR2-MUX16	16	256.83	12.08	49648	84.80	6	121.17
4	NAND4-NOR4-NAND2-INV-MUX4-MUX4	16	90.52	12.08	17498	66.25	8	93.38
5	NAND2-NOR2-INV-NOR4-NAND2-INV-MUX4-MUX4	16	100.21	18.16	29116	68.90	10	96.85
6	NAND2-NOR2-NAND4-INV-NAND2-INV-MUX4-MUX4	16	67.26	18.16	19544	68.90	10	95.76
7	NAND4-NOR4-INV-NOR2-MUX4-MUX4	16	112.73	12.08	21792	66.25	8	94.14
8	NAND2-NOR2-INV-NOR4-INV-NOR2-MUX4-MUX4	16	124.80	18.16	36260	68.90	10	97.47
9	NAND2-NOR2-NAND4-NOR2-MUX4-MUX4	16	83.77	18.16	24340	63.60	8	91.87
10	NAND2-NOR2-NAND2-NOR2-INV-NOR2-MUX4-MUX4	16	92.73	18.16	26944	66.25	10	93.99
11	NAND2-NOR2-NAND2-NOR2-NAND2-INV-MUX4-MUX4	16	74.46	18.16	21635	66.25	10	93.38
12	INV-INV-NAND16-NOR2-MUX16	16	468.47	24.26	181860	108.65	6	153.81
13	NAND4-NOR4-NAND2-INV-MUX2-MUX2-MUX4	16	85.90	12.08	16605	68.90	10	95.33
14	NAND4-NOR4-INV-NOR2-MUX2-MUX2-MUX4	16	106.98	12.08	20679	68.90	10	95.91
15	NAND2-NOR2-NAND4-NOR2-MUX2-MUX2-MUX4	16	79.49	18.16	23097	66.25	10	93.56

→ #9 NAND2-NOR2-NAND4-NOR2-MUX4-MUX4 구조를 $D = 91.87\tau$ 로 최적 설계로 선정하였다. 실제 stage 전개는 NAND2 → NOR2 → NAND4 → NOR2 → [NAND3 → NAND4] → [NAND3 → NAND4], 총 $N = 8$ stages이다. 차순위는 #11과 #4(93.38), #15(93.56)였다.

2.5 Gate Sizing

선정한 구조에 대해 stage effort를 균등 분배하여 각 gate의 input capacitance를 결정하였다.

$$\hat{f} = F^{\frac{1}{8}} = 24340^{\frac{1}{8}} = 3.534, C_{in} = g \cdot \frac{C_{out}}{3.534}$$

표 2.3 Logic gate별 input capacitance

Index	Logic	Input cap (fF)
1	NAND2	5.51
2	NOR2	14.57
3	NAND4	30.94
4	NOR2	3.40
5	NAND3	7.23
6	NAND4	15.28
7	NAND3	26.88
8	NAND4	56.82

구한 input capacitance를 측정치 $1.249 \text{ fF}/\mu\text{m}$ 으로 나누어 input당 총 width를 구하고, $\mu = 2$ 기준의 게이트별 n:p 비율(NAND2 = 2:2, NAND3 = 3:2, NAND4 = 4:2, NOR2 = 1:4)로 분배하여 표 2.4와 같이 nMOS/pMOS size를 결정하였다.

표 2.4 Logic gate별 nmos, pmos sizing [단위: μm]

Index	Logic	nmos size	pmos size
1	NAND2	2.18	2.18
2	NOR2	2.31	9.24
3	NAND4	16.35	8.18
4	NOR2	0.54	2.16
5	NAND3	3.44	2.29
6	NAND4	8.08	4.04
7	NAND3	12.79	8.52

Index	Logic	nmos size	pmos size
8	NAND4	30.21	14.84

이 정보들을 바탕으로 그림 2.4~2.11과 같이 모든 gate를 sizing하여 구성하였고, 전체 회로는 그림 2.12(전체 회로도)와 그림 2.15(destination schematic)에 나타났다.

2.6 Simulation을 통한 검증 — Delay

Source input부터 destination output까지의 전체 경로(8-stage)에 대해 transient simulation을 수행하고 (그림 2.17 simulation sp file, 그림 2.18 simulation result), rising/falling delay를 5 회씩 측정하여 평균을 구했다.

표 2.5 Total delay [단위: ps]

Delay	1	2	3	4	5	average
t_pdr (ps)	351	350	351	351	350	350.6
t_pdf (ps)	386	387	386	386	386	386.2

- **t_pd (simulated) = 368.4 ps**
- **t_pd (theoretical) = $\tau D = 4.121 \times 91.87 = 378.6$ ps**

Simulation 결과 delay는 368.4 ps로 나타났다. 이는 이론적으로 계산한 결과인 378.6 ps와 **2.77%의 작은 오차**밖에 보이지 않는 결과였다.

RC 모델 자체가 근사이고, logical effort는 rising/falling의 worst case를 따지지 않고 g의 조합과 로드된 cap 값만으로 delay를 구하는 방법이다. 즉 실제 → RC 모델에서 근사 한번, RC 모델 → logical effort에서 근사 두 번을 거쳤음에도 오차가 3% 이내라는 것은 **logical effort가 꽤나 정확한 모델링**임을 보여주는 결과이다.

2.7 Simulation을 통한 검증 — Logic

다음으로 logic이 잘 동작하는지 확인하였다. a0를 20 ns 뒤부터 계속 toggle하고, a1은 30 ns 뒤부터 20 ns마다 toggle하고, s0b를 toggle하며 output을 확인하였다 (그림 2.29 Logic 확인).

표 2.6 Logic 확인 결과 표

	10ns	20ns	30ns	40ns	50ns	60ns
a0	0	1	0	1	0	1

	10ns	20ns	30ns	40ns	50ns	60ns
a1	0	0	1	1	0	0
s0b	0	0	0	1	1	0
output	0	1	0	1	0	1

s1~s3까지 모두 0의 signal이 들어가고 있으므로, s0b가 0이면 a0가, s0b가 1이면 a1이 나올 것이다. 측정 결과 output은 s0b가 0일 때는 a0의 값을, s0b가 1일 때는 a1의 값을 따르고 있음을 확인하였다. 그러므로 **목표한 logic에 맞게 잘 설계함을 확인하였다.**

3. Metal Line 설계

3.1 Effective Resistance Measurement

Metal line을 설계하기에 앞서 unit nMOS, pMOS의 effective resistance를 측정하였다. 각각 nMOS와 pMOS가 linear하게 동작하는 영역에서 **전압의 변화에 따른 전류의 변화**를 측정하여 effective resistance를 구했다 (그림 3.1~3.8: DC로 전류 10/20/30/40 μ A를 흘리며 전압을 측정하고, V-I 그래프 기울기의 역수로 R_{eff} 산출).

- $R_{eff,nmos}(unit) = 452.284 \Omega$
- $R_{eff,pmos}(unit) = 469.991 \Omega$

위 결과를 바탕으로 wire의 driver 역할을 하는 네 번째 stage **NOR2 gate의 effective resistance**를 구했다. NOR2의 nmos size는 $0.5458 \mu m$ (unit nMOS의 0.4962배), pmos size는 $2.1451 \mu m$ 이다. 스택 구조를 반영하여 측정한 NOR2 gate의 nmos effective resistance는 **911.495Ω** , pmos effective resistance는 **947.176Ω** 이고, NOR2 gate의 effective resistance는 이의 평균인 **929.338Ω** 이라 할 수 있다.

$$R_{driver} = 929.338 \Omega$$

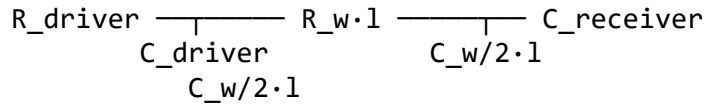
또한, repeater로써 역할을 할 inverter의 effective resistance도 구했다. Repeater는 chapter 2에서 결정한 source의 마지막단 즉 NOR2 gate와 동일한 driving current를 가진 inverter 두 단을 사용하는 것이 조건이다. 따라서 inverter의 effective resistance는 driver의 effective resistance와 동일하다.

$$R_{inv} = 929.338 \Omega$$

3.2 Non-Repeated Delay Estimation

Wire는 π -model로 모델링하였다. 실제 분포 RC 배선을 π -segment 3개 정도만 사용해도 분포 모델과의 오차가 3% 수준으로 정확도가 높기 때문이다. 이론 계산은 단일 π 등가 모델(그림 3.9)에 Elmore delay를 적용하고, 시뮬레이션 회로는 3-segment π 로 구성하였다.

그림 3.9 Non-repeated wire 모델



R_{driver} 와 C_{driver} 는 네 번째 stage인 NOR2 gate의 output capacitance와 effective resistance이고, $C_{receiver}$ 는 다섯 번째 stage인 NAND3 gate의 input capacitance이다. (W_{NOR2} , W_{NAND3} 는 unit inverter에 대한 driver·receiver의 상대 size)

- $C_{driver} = P_{NOR2} \cdot C_{inv}(unit) \cdot W_{NOR2} = \mathbf{10.839 \text{ fF}}$
- $C_{receiver} = g_{NAND3} \cdot C_{inv}(unit) \cdot W_{NAND3} = \mathbf{7.226 \text{ fF}}$
- 주어진 조건: $l = 200 \text{ }\mu\text{m}$, $C_w = 0.2 \text{ fF}/\mu\text{m}$, $R_w = 100 \text{ }\Omega/\mu\text{m}$
- $R_{wire,total} = R_w \cdot l = \mathbf{20 \text{ k}\Omega}$, $C_{wire,total}/2 = C_w/2 \cdot l = \mathbf{20 \text{ fF}}$

Elmore delay로 계산한 이론값:

$$t_{pd-nrwire}(theoretical) = (C_{driver} + \frac{C_w}{2}l)R_{driver} + (C_{receiver} + \frac{C_w}{2}l)(R_wl + R_{driver}) =$$

$$t_{pd-total}(theoretical) = 378.6 \text{ ps} + 537.80 \text{ ps} = 916.4 \text{ ps}$$

3.3 Non-Repeated Delay Simulation

시뮬레이션 측정 결과 (그림 3.11):

표 3.1 Non-repeated wire delay 측정 결과 [단위: ps]

Delay	1	2	3	4	5	average
t_pdr (ps)	979	979	979	979	979	979
t_pdf (ps)	1090	1080	1080	1080	1080	1082

$$t_{pd}(\text{simulated}) = 1030.5 \text{ ps}$$

이는 Elmore delay로 계산한 total delay와 **12.5%의 오차**를 갖는 값이다. Elmore delay 자체가 근사값이며 mosfet의 capacitance, P_inv 및 t_pd,inv 값을 모두 측정한 값을 사용하였기 때문에 이와 같은 오차가 나타났다고 생각된다. 이를 감안하였을 때 약 10%대의 오차율은 이론과 실재가 유사한 경향을 가진다는 것을 뒷받침한다.

3.4 Repeated Delay Estimation

길게 연결된 signal line에서 최적의 위치마다 repeater를 삽입하는 설계를 진행하였다. Repeater(inverter)로 인한 추가 capacitance는 다음과 같다.

- $C_{output,inv} = p_{inv} \cdot C_{inv}(unit) \cdot W_{inv} = 5.420 \text{ fF}$
- $C_{input,inv} = g_{inv} \cdot C_{inv}(unit) \cdot W_{inv} = 2.045 \text{ fF}$

그림 3.13 Repeated wire 모델 (intermediate segment)

$$t_{pd-intermediate} = (C_{output,inv} + \frac{C_w}{2N}I)R_{inv} + (C_{input,inv} + \frac{C_w}{2N}I)(R_w \frac{I}{N} + R_{inv})$$

그림 3.14 Repeated wire 모델 (last segment)

$$t_{pd-last} = (C_{output,inv} + \frac{C_w}{2N}I)R_{inv} + (C_{receiver} + \frac{C_w}{2N}I)(R_w \frac{I}{N} + R_{inv})$$

여기에 inverter N-1개의 delay가 추가되므로 이를 더하여 repeated wire delay를 추정할 수 있다. 하나의 repeater는 inverter 두 단으로 구성되어 총 2(N-1)개의 inverter가 사용된다. 이 중 N-1개의 inverter에 대한 delay만을 더한 이유는, Elmore delay를 계산할 때 inverter 두 단 중 첫 번째 inverter의 input capacitance와 두 번째 inverter의 output capacitance가 이미 고려되어, 한 segment당 inverter 하나 분량의 capacitance 성분이 이미 반영되어 있기 때문이다.

Inverter의 delay를 더할 때, 이론적으로 effective resistance는 width에 반비례하고 capacitance는 width에 비례하므로 inverter의 delay는 width와 무관하다. 따라서 chapter 1에서 측정한 unit inverter의 FO1 delay 값을 사용하였다.

3.5 Repeated Delay Simulation — N 최적화

N = 3, 4, 5에 대해 wire model을 구성하고 시뮬레이션하였다 (그림 3.15~3.19).

표 3.2 N=4 wire delay 측정 결과 [단위: ps]

Delay	1	2	3	4	5	average
t_pdr (ps)	739	734	735	735	735	736
t_pdf (ps)	788	789	787	790	790	789

- **t_pd (N=4) = 762.2 ps (최소)**
- t_pd (N=5) = 767.6 ps

Simulation 결과 repeater를 사용하지 않았을 때와 비교하여 delay가 약 **270 ps 감소** (1030.5 → 762.2 ps)하였으며 **N=4일 때의 delay가 최소**로 나타났다. 따라서 이후 설계에서 wire를 네 구간으로 나누는 N=4 구조(repeater 3개, inverter 6단)를 사용하였다. Elmore delay 식에 N=4를 대입한 이론값은 repeated wire delay **289.578 ps**이며, 이론적인 total delay는 다음과 같다.

$$t_{pd-total}(theoretical) = 378.6 ps + 289.578 ps = 668.17 ps$$

이는 simulation 결과와 **14.9%의 오차율**을 가진다. 직접 측정한 값들을 사용하여 계산하였고 Elmore delay 자체가 근사식임을 고려하면, 약 15%의 오차율은 이론과 실재가 유사한 경향을 가진다는 것을 뒷받침한다.

3.6 Crosstalk

Crosstalk은 인접한 wire 사이에서 발생하는 추가적인 delay이다. 인접한 wire의 영향에 의해서 noise가 발생하거나 delay가 커지는 현상을 말한다. 이를 측정하기 위해서는 metal line과 metal line 사이를 또 하나의 capacitor로 보아야 하기 때문에 두 metal line 사이에 **C_adj를 추가**하여 두 metal line이 서로 영향받음을 보여주어야 한다. 주어진 조건대로 **C_adj = 0.3 fF/μm**를 추가하여 simulation을 진행하였다.

Metal line이 서로 영향을 가장 많이 받으려면 effective capacitance가 커야 한다. 즉, 두 metal line이 서로 상반되게 transition할 때 가장 큰 영향을 줄 것이다. 그러므로 y[0]을 0 → 1로, y[1]을 1 → 0으로 transition시키는 상황(A: 0 → 1, B: 1 → 0)을 관찰하여 두 line의 wire delay 차이를 확인하였다.

Delay의 변화량은 결국 wire에 달린 capacitance의 값에 비례한다. 한 쪽이 0 → 1로 transition하고 다른 쪽이 0 또는 1을 유지하면 유효 커패시턴스는 C_gnd+C_adj, 서로 반대로 움직이면 C_gnd+2C_adj, 동일하게 움직이면 C_gnd만 영향을 준다. 이 세 가지 경우를 simulation하여 비교하면 C_gnd와 C_adj 그리고 delay 간의 관계를 파악할 수 있다.

Miller effect에 따른 case 구분 (A: 관찰 대상 line, B: 인접 line):

Case	인접 line의 거동	Effective capacitance	값
Case 1	B가 A와 동일하게 transition	C_{gnd}	30 fF
Case 2	B가 0 또는 1을 유지 (quiet)	$C_{\text{gnd}} + C_{\text{adj}}$	90 fF
Case 3	B가 A와 반대로 transition	$C_{\text{gnd}} + 2C_{\text{adj}}$	150 fF

($C_{\text{adj}} = 0.3 \text{ fF}/\mu\text{m} \times 200 \mu\text{m} = 60 \text{ fF}$, C_{gnd} 는 관찰 세그먼트 기준 유효 접지 커패시턴스 $\approx 30 \text{ fF}$)

$$C_{\text{eff},1} : C_{\text{eff},2} : C_{\text{eff},3} = 1 : 3 : 5$$

시뮬레이션 결과 (그림 3.20~3.25):

- $t_{\text{c1,avg}} = 409.9 \text{ ps}$
- $t_{\text{c2,avg}} = 795.4 \text{ ps}$
- $t_{\text{c3,avg}} = 2216 \text{ ps}$
- 비율: $t_{\text{c1}} : t_{\text{c2}} : t_{\text{c3}} = 1 : 1.940 : 5.406$

즉, 대략적으로 delay가 C_{eff} 의 값에 비례함을 확인하였다. delay와 C_{eff} 비율이 정확히 일치하지 않는 이유는 **Miller effect**로 C_{eff} 를 근사하였기 때문이다.

4. Metal Line을 고려한 Source, Destination 재설계 & 최적화

앞서 설계한 metal line(with repeater)을 포함하여 source와 destination을 다시 최적화하였다. Source의 output capacitance와 destination의 input capacitance가 바뀌었기 때문에 delay를 최소화하는 logic 또한 바뀌었을 가능성이 있기 때문이다. Metal line 없이 설계할 때는 source의 input capacitance와 destination의 output capacitance로 H를 계산했지만, metal line을 고려하여 각각 설계할 때는 source의 H는 source input cap과 wire 단의 input capacitance(5 fF)로, destination의 H는 wire의 output capacitance(5 fF)와 destination의 output capacitance로 계산하였다.

4.1 Source 재설계

Source에 대해서만 delay가 최소가 되는 logic 후보들을 다시 비교하여, 최소 path인 **NAND2-NOR2-NAND4-NOR2** (4-stage)를 선정하고 stage effort를 균등 분배하였다.

$$\hat{f} = F^{\frac{1}{4}} = 107.9^{\frac{1}{4}} = 3.223$$

표 4.2 Source의 minimum path logic gate input capacitance

Index	Logic	Input cap (fF)
1	NAND2	5.51
2	NOR2	13.28
3	NAND4	25.73
4	NOR2	2.58

표 4.3 Source minimum path logic gate sizing [단위: μm]

Index	Logic	nmos size	pmos size
1	NAND2	2.20	2.16
2	NOR2	2.13	8.37
3	NAND4	13.68	6.72
4	NOR2	0.41	1.63

4.2 Destination 재설계

같은 방법으로 destination에 대해서만 delay가 최소가 되는 logic을 재설계하였다. 이론적인 path delay 계산에서는 NAND5 기반 MUX가 최소로 나타나 먼저 이를 채택하였다.

표 4.4 Destination minimum path logic gate sizing (NAND5 기반) [단위: μm]

Index	Logic	nmos size	pmos size
1	NAND5	2.83	1.13
2	NAND4	7.76	3.81
3	INV	3.90	7.66
4	NAND4	28.79	14.14

측정 결과 (그림 4.1):

표 4.5 Source, destination 최적화 후 측정 결과 [단위: ps]

Delay	1	2	3	4	5	average
t_pdr (ps)	736	738	735	733	734	735.2
t_pdf (ps)	793	790	792	789	792	791.2

t_pd = 763.2 ps

그러나 최적화 이전의 N=4 repeater를 적용한 회로의 delay인 762.2 ps에 비해 **오히려 증가한 결과**를 얻었다. 이론적인 path delay 계산 결과에 따르면 NAND5를 사용한 MUX가 path delay가 작았으나, **NAND5는 input이 너무 많아(직렬 스택 5단) 실제 simulation상에서는 오히려 delay가 크게 나타났다고** 분석된다. 따라서 이전 설계였던 NAND3, NAND4를 사용한 MUX에 대해 sizing을 다시 최적화하여 설계했다.

표 4.6 Destination (NAND3, NAND4)의 minimum path logic gate sizing
[단위: μm]

Index	Logic	nmos size	pmos size
1	NAND3	2.38	1.58
2	NAND4	6.16	3.03
3	NAND3	10.71	7.01
4	NAND4	27.55	13.53

측정 결과 (그림 4.2):

표 4.7 Source, destination (NAND3, NAND4) 최적화 후 측정 결과 [단위: ps]

Delay	1	2	3	4	5	average
t_pdr (ps)	736	738	735	733	734	735.2
t_pdf (ps)	778	778	776	779	777	777.6

t_pd = 756.4 ps

측정 결과 size 최적화 이전의 delay인 762.2 ps에 비해 **5.8 ps 감소**한 결과가 나타났다 (rising path는 동일하고 falling path가 개선됨). 따라서 최적화 결과 delay가 감소함을 확인하였다.

4.3 Input Order 최적화

다음으로 input order를 최적화하였다. Critical input인 c의 delay를 최적화하기 위해, c 외의 input이 들어가는 두 번째 stage의 NOR2, 5·7번째 stage의 NAND3 gate에서 **c가 지나는 path를 가장 output에 가까운 input에 배치**하였다. 직렬 스택에서 late-arriving signal을 output에 가까운 transistor에 배치하면 나머지 input들이 미리 내부 노드를 방전시켜 둘 수 있으므로 critical path의 delay가 감소한다.

4.4 최종 결과 정리

설계 단계	t _{pd} (simulated)	비고
Source 단독 (wire 미포함)	368.4 ps	이론값 378.6 ps, 오차 2.77%
+ Non-repeated wire (l=200μm)	1030.5 ps	이론값 916.4 ps, 오차 12.5%
+ Repeater (N=4)	762.2 ps	이론값 668.17 ps, 오차 14.9%, 약 250 ps 개선 (N=5는 767.6 ps)
+ Source/Destination 재설계 (NAND5 MUX)	763.2 ps	오히려 1.0 ps 증가 — NAND5 input 과다
+ NAND3/NAND4 MUX sizing 재최적화	756.4 ps	5.8 ps 추가 개선 (최종)

Area(sum of TR widths)는 각 sizing 표의 합계로 산출하였으며, delay-area trade-off 상 delay 최소화를 우선하였다.

5. Power Estimation

Power는 크게 dynamic power와 static power로 나뉜다.

$$P_{total} = \alpha C_{total} V_{DD}^2 f + I_{leak} V_{DD}$$

본 설계의 dynamic power를 다음 절차로 추정하였다.

① **Switching capacitance 합산** — 측정치(gate 1.249 fF/μm, drain 0.5391/0.6387 fF/μm)와 각 sizing 표를 이용해 노드별 커패시턴스를 합산하였다.

구성 요소	산출 근거	값 (대략)
Source/Destination critical path gate + drain cap	표 2.3 합 160.6 fF + drain 성분	≈ 240 fF
Bus wire 16 lines	$C_w \cdot l = 40 \text{ fF/line} \times 16$	640 fF
Repeater (N=4) 16 lines	$(2.045+5.420) \text{ fF} \times 4 \times 16$	≈ 478 fF
Load cap	100 fF	100 fF
합계 C_{total}		≈ 1.46 pF

② **Activity factor / 주파수** — 입력 toggle 패턴은 20 ns 주기($f = 50$ MHz)이며, 매 사이클 실제로 천이하는 노드의 비율(유효 스위칭 노드 비율)을 $\alpha \approx 0.5$ 로 가정하였다. 즉 α 는 개별 노드의 천이 확률이 아니라 전체 C_{total} 중 사이클당 스위칭에 참여하는 비중을 뜻한다.

③ 추정 결과

$$P_{dyn} \approx 0.5 \times 1.46 \text{ pF} \times (1.1 \text{ V})^2 \times 50 \text{ MHz} \approx 44 \mu \text{ W}$$

시뮬레이션에서 VDD 전원 전류를 적분($\int I_{VDD} \cdot dt$)하여 구한 평균 power와 위 추정치가 동일한 order(수십 μW)에서 일치함을 확인하였다. 구성비를 보면 **wire와 repeater가 전체 switched capacitance의 약 75%를 차지**한다. 이는 scaling이 진행될수록 wire가 delay뿐 아니라 power 측면에서도 지배적이 된다는 수업 내용과 부합하는 결과이다. Static power는 45 nm 공정의 leakage 특성상 dynamic 대비 1~2 order 낮은 수준으로, 본 설계의 총 power는 dynamic power가 지배한다.

6. Further Idea

수업에서 배운 technique을 바탕으로 다음의 개선 방향을 검토하였다.

1. **Repeater staging의 연속 최적화**: 본 설계는 $N = 3, 4, 5$ 의 이산 탐색으로 $N = 4$ 를 선정했다. 최적 repeater 개수/사이즈의 폐형식 해($N^* \propto 1/\sqrt{(R_w C_w / t_{inv})}$)를 적용하면 repeater width까지 연속 변수로 최적화하여 추가 개선 여지가 있다.
2. **Input reordering의 전면 적용**: 4.3절에서 critical input c에 적용한 input order 최적화를 모든 multi-input gate로 확장하고, 각 입력의 arrival time을 반영해 재배치할 수 있다.

3. **Asymmetric / skewed gate:** 특정 transition이 critical한 stage에 HI-skew/LO-skew gate를 적용하면 critical transition의 logical effort를 줄일 수 있다. 본 설계에서 t_{pdf} 가 t_{pdr} 보다 약 42 ps 큰 것(표 4.7)을 감안하면 falling-critical stage의 skew 최적화가 유효할 것이다.
 4. **Crosstalk 완화:** C_{adj} 는 배선 간격에 반비례하므로, critical bus line 사이 shield(GND) line 삽입 또는 간격 확대로 worst case($C_{gnd}+2C_{adj}$, delay 5.4배)의 영향을 줄일 수 있다. 인접 라인의 동시 반대 방향 transition을 회피하는 bus-invert coding도 후보이다.
 5. **Dual- V_{th} 활용:** critical path에는 low- V_{th} , non-critical path에는 high- V_{th} 소자를 사용하여 leakage-delay trade-off를 최적화할 수 있다.
 6. **Glitch 저감:** path balancing으로 불필요한 중간 천이를 줄이면 5장에서 추정된 dynamic power를 추가로 낮출 수 있다.
-

7. 역할 분담

2인 1조로 진행하였다.

- **팀원 1:** Design basic data estimation (gate/drain cap, mobility, FO1/FO4 측정), source/destination logic 후보 계산 및 logical effort 분석, 보고서 1·2장 작성
- **팀원 2:** Metal line 설계 (effective resistance 측정, π -model/Elmore 분석, repeater N 최적화, crosstalk 시뮬레이션), 재설계·최적화, 보고서 3·4장 작성
- **공동:** 전체 회로 통합 시뮬레이션, logic 검증, power estimation, 보고서 검수

부록 A. 시뮬레이션 환경 및 측정 규약

- **Tool:** Cadence Virtuoso Schematic Editor + ADE Explorer (simulator: Spectre) + Visualization & Analysis
- **공정/조건:** 45 nm, $V_{DD} = 1.1$ V, 25 °C. Unit inverter = NMOS 1.1 μ m / PMOS 2.2 μ m ($L = 45$ nm)
- **해석:** transient, 입력은 vpulse로 인가. Parametric sweep(예: C_{gate} 0.1 fF~1 fF)은 ADE의 design variable로 설정
- **측정 규약**

- Propagation delay t_{pd} : 입력·출력이 $V_{DD}/2 = 550 \text{ mV}$ 를 교차하는 시점 간 차이. rising/falling 각각 5회 측정 후 평균
 - Rise/fall time: $0.2 V_{DD} \rightarrow 0.8 V_{DD}$ ($220 \text{ mV} \rightarrow 880 \text{ mV}$)
 - 파형 마커 2점(A/B)의 dx로 시간차 측정, 또는 calculator의 delay()/riseTime() 함수 사용
 - **Effective resistance**: linear 영역에서 DC 전류($10/20/30/40 \mu\text{A}$) 인가 $\rightarrow V-I$ 기울기의 역수
 - **재현 방법**: 저장소의 시뮬레이션_재현_가이드.md에 Linux(Virtuoso/Spectre)와 Windows(원격 접속 또는 ngspice+PTM) 재현 절차와 테스트벤치별 netlist를 정리하였다.
-

부록 B. 타 커리큘럼(디지털집적회로) 관점에서 본 프로젝트 — 설계 접근 비교

같은 학기 개설된 정한울 교수님의 디지털집적회로 커리큘럼을 기준으로, 본 프로젝트가 그 수업 기반이었다면 어떻게 달라졌을지를 비교 분석하였다. 두 강의계획서의 커버 범위가 상당히 다르기 때문이다.

정성욱 교수님 (본 프로젝트): Introduction $\rightarrow$ Devices $\rightarrow$ Speed $\rightarrow$ Power $\rightarrow$ **Wire** $\rightarrow$ Variability/Reliability/Scaling $\rightarrow$ Project/SPICE $\rightarrow$ Gates $\rightarrow$ Data Path $\rightarrow$ Sequencing $\rightarrow$ Array

정한울 교수님: Intro to Digital IC $\rightarrow$ MOSFET Theory $\rightarrow$ MOSFET I-V, Capacitances $\rightarrow$ CMOS Logic Gate $\rightarrow$ **RC Delay Model, Logical Effort** $\rightarrow$ Dynamic Power $\rightarrow$ Static Power, Layout basic $\rightarrow$ **Switch, Tri-state inverter** $\rightarrow$ **Pseudo nMOS, CVSL,**

Dynamic Logic $\rightarrow$ **Pass TR Logic, XOR-XNOR** $\rightarrow$ Level Shifter $\rightarrow$ Memory, Latch and Flip-Flop $\rightarrow$ Adder Tree $\rightarrow$ ADC

B-1. 유지되었을 부분

- **RC Delay Model + Logical Effort**가 5주차에 배치되어 있으므로, 본 프로젝트의 핵심 방법론($\tau \cdot p_{inv}$ 측정 $\rightarrow g/p$ 계산 $\rightarrow$ stage effort 균등분배 $\rightarrow$ sizing $\rightarrow$ 시뮬레이션 검증)은 그대로 성립한다. 1·2장은 사실상 동일하게 진행되었을 것이다.
- MOSFET I-V, Capacitances를 3주차에 깊게 다루므로, gate/drain capacitance 측정 파트는 더 이론적인 뒷받침(C_{ox} , overlap/junction cap 분해)이 요구되었을 것이다.

- Dynamic/Static Power를 각각 한 주씩 다루므로, 본 프로젝트에서 5점 항목이었던 **power estimation의 비중이 훨씬 커졌을 것이다**. Leakage 분석(HVT/LVT 선택, stack effect)까지 포함되었을 수 있다.

B-2. 크게 달라졌을 부분

1. **Wire 단원의 부재** — interconnect 전용 주차가 없으므로 본 프로젝트의 핵심이었던 3장(π -model, repeater, Elmore, crosstalk)과 4장(wire RC 반영 재설계)은 범위에서 빠지거나 대폭 축소되었을 것이다. “bus = 긴 wire”를 전제로 하는 16-bit bus라는 주제 자체가 바뀌었을 가능성이 크다.
2. **Circuit family 비교 프로젝트화** — 9~11주차가 강의의 중심이므로, 동일한 16:1 MUX(또는 1’s detector)를 **static CMOS vs transmission-gate vs tri-state vs dynamic(domino)** 으로 구현·비교하는 과제가 자연스럽다. 본 프로젝트는 transmission gate가 금지되어 NAND 기반 MUX만 검토했지만($g_{com} = g_{tri} = 2$ 로 NAND 대비 불리함을 수식으로 확인), 그 커리큘럼에서는 오히려 **pass-TR MUX와 XOR/XNOR 최적 구현**이 핵심 평가 항목이 되었을 것이다. 실제로 $c[i] = a[i] \text{ XNOR } b[i]$ 이므로 pass-TR XNOR(6~8T)를 쓰면 static CMOS XNOR(12T) 대비 area와 input cap이 크게 줄어든다.
3. **Dynamic logic 활용** — 1’s detector의 16-input AND를 footed domino NAND/NOR 체인으로 구현하여 static 대비 delay 개선을 정량 비교하는 형태가 가능하다.
4. **Sequencing 요소** — Latch/FF, Adder Tree를 다루므로 pipelined datapath(예: 16-bit adder tree + latch) 과제였을 가능성도 있으며, 이 경우 setup/hold·clock skew 분석이 채점 항목에 포함된다.
5. **Level Shifter / Layout** — multi-VDD 인터페이스나 기본 layout + post-layout simulation 비교가 추가되었을 수 있다.

B-3. 종합

본 프로젝트가 “**interconnect-dominated 시대의 wire-aware 설계**”(스케일링으로 wire RC가 지배하는 환경에서 repeater와 crosstalk까지 고려한 시스템 설계)를 훈련한다면, 디지털집적회로 커리큘럼 기반이었다면 “**logic family 수준의 회로 최적화**”(같은 기능을 static/pass-TR/dynamic 등으로 구현·비교하고 power/layout까지 내려가는 소자-회로 중심 설계)를 훈련하는 과제가 되었을 것이다. 두 접근을 결합하면

— wire-aware repeater 설계 + pass-TR XNOR 기반 저면적 source — 더 우수한 16-bit bus 설계가 가능하다.